

Poetry Commands Guide for Python Execution

Poetry is a powerful tool for dependency management and packaging in Python. This guide focuses on the commands you need to run scripts, modules, and manage environments effectively.

1. Running Scripts and Modules

These are the most common commands for executing your code within the Poetry-managed virtual environment.

Run a Specific File

Executes a single Python script within the project's virtual environment.

```
poetry run python path/to/your_script.py
```

Run a Module

Executes a module with the `-m` flag (useful if your project relies on relative imports or is structured as a package).

```
poetry run python -m my_module
```

Running Custom Scripts (Defined in `pyproject.toml`)

If you define scripts in your `pyproject.toml` file, you can run them by name. Example configuration:

```
[tool.poetry.scripts]
start-app = "my_package.main:run"
```

Command:

```
poetry run start-app
```

2. Environment Shell and Activation

Sometimes you want to enter the virtual environment directly rather than prefixing every command with `poetry run`.

Spawn a Shell

Activates the virtual environment in your current terminal session.

```
poetry shell
```

Get Environment Info

Check where the virtual environment is located and which Python version it is using.

```
poetry env info
```

Exit the Shell

Simply type `exit` to leave the virtual environment and return to your standard shell.

```
exit
```

3. Dependency and Module Management

Before you can run your code, you need to ensure your modules and dependencies are correctly installed.

Initialize a New Project

```
poetry init # Interactive setup for an existing directory
poetry new my-project # Creates a new folder structure
```

Add a New Module/Dependency

```
poetry add requests
poetry add pytest --group dev # Add to development dependencies
```

Install Project Dependencies

Installs all modules listed in `pyproject.toml` .

```
poetry install
```

Update Modules

```
poetry update          # Updates all dependencies to latest compatible versions
poetry update requests # Updates a specific package
```

4. Troubleshooting and Maintenance

List Installed Packages

```
poetry show
poetry show --tree # Visualizes dependency hierarchy
```

Remove a Dependency

```
poetry remove package-name
```

Check for Security Vulnerabilities

(Requires plugin or specific versions)

```
poetry check
```

Clear Cache

If you encounter weird installation issues:

```
poetry cache clear pypi --all
```

Pro Tip: Poetry "No-Interaction"

When running poetry in CI/CD pipelines (like GitHub Actions), add the `--no-interaction` flag to prevent the command from hanging on user prompts:

```
poetry install --no-interaction
```